Corporate WINLine®
Object Model

Valid From
Version
Corporate WINLine® 10.5

Table of Contents

1. Object Hierarchy

- **CWLStart**
  - CurrentCompany (CWLCompany)
  - **CWLCurrentcompany**
    - Module (CWLModul)
    - CurrentWindow (CWLWindow)
    - **CWLModul**
      - CurrentWindow (CWLWindow)
      - Windows (CWLWindowCollection)
        - Item (CWLWindow)
        - NamedItem (CWLWindow)
        - IndexedItem (CWLWindow)
      - **CWLWindow**
        - Vars (CWLWindowVars)
        - **CWLWindowVars**
          - Controls (CWLFgCollection)
            - Item (CWLFgControl)
            - IndexedItem (CWLFgControl)
          - CurrentControl (CWLFgControl)
          - **CWLFgControl**
            - Preview (CWLPreview)
            - **CWLPreview**
              - Page (CWLPreviewPage)
              - **CWLPreviewPage**
                - Item (CWLPreviewPageItem)
                - **CWLPreviewPageItem**
            - SpreadSheet (CWLSpreadSheet)
            - **CWLSpreadSheet**
            - Bildschirmtabelle (CWLGrid)
            - **CWLGrid**
        - **CWLReport**
- **CWLCurrentModule**
  - ActiveModule (CWLModule)
- **CWLCurrentWindow**
  - ActiveWindow (CWLWindow)
- **CWLScript**
- **CWLMacro**
- **CWLEventResult**
- **GeneralScriptFuncs**
- **LOHNFormel**
- **FAKTFormel**

2. Objects

The following inherent objects can be used in CWL VBScript:

Object name	**Use**
CWLCurrentModule	Only available in CTK scripts. Its sole property **ActiveModule** corresponds to **CWLStart.CurrentModule**. The object is the event interface for module-specific events. It represents at any moment the currently active module.
CWLCurrentWindow	Only available in CTK scripts. Its sole property **ActiveWindow** corresponds to **CWLStart.CurrentWindow**. The object is the event interface for dialogue-specific events. It corresponds at any moment the currently active dialogue window.
CWLScript	Represents the script. Obejct is available in Payroll scripts, CTK scripts and System scripts.
CWLStart	Serves to control the application. Available only in system scripts and CTK scripts.
FormDriver	Used only internally.
MacroCommands or CWLMacro	Performs all important functions for macro processing. Can be used in all scripts.
UserForm	Represents the UserForm. Is available in all scripts which use a UserForm: CTK scripts, System scripts, Payroll scripts. It can be used to execute direct actions with the script window (e.g., reaction to mouse clicks in the window)
CWLWindowVars	Access to window variables
CWLEventResult	Return of results in events which support the object
CWLSearchResult	Contains the result of an SQL query (see CWLCompany - Object)
GeneralScriptFuncs	Support for MsgBox and InputBox for scripts in the EWL (VBScript functions are executed in the EWL on the server, not on the EWL client). With FileDialog you can specifiy files and with WaitCursor you can display the timer (sand clock).
CWLTable	Represents an opened table. Object is supported by CWLDbConnection.
CWLReport	Use this object to program your own reports in the CWL. The object can be created by a CWLWindow.

3. MacroCommands

This object is used mainly in macro processing, since it is also available in the macro recorder. Some of the functions which are available with this object can also be realized with other CWL objects.

Accessible in

- ☐ System Macros
- ☐ CTK Macros
- ☐ Macro Recorder
- ☐ LOHN Macros
- ☐ FAKT Macros

4. Object Model Descriptions

4.1. CWLStart

This object serves to control and steer the entire application.

Exposed in

☐ System Macros
☐ CTK Macros

Attention: This object is the default object in these scripts and all properties and methods can be used without the CWLStart prefix.
Name is for example not a user-defined variable, but rather is the same as **CWLStart.Name** (except, of course, when you define **Name** with **Dim**).

CWLStart

Properties

ICwlStart* **Application**
BSTR **FullName**
BSTR **Name**
BOOL **Visible**
ICWLModule* **CurrentModule**
ICwlWindow* **CurrentWindow**
BSTR **AppPath**
BSTR **WorkPath**
BSTR **ServerPath**
ICWLCompany * **CurrentCompany**
BSTR **GlobalProperty** (int PropertyNr)
ICWLDbConnection ***Connection**

Methods

ICWLModule* **Module** (short nApplicationNr)
void **Quit** ()
BOOL **ActivateModule** (CWLApplicationNr nApplicationNr)
BOOL **ActivateExternalApp** (short nApplicationId)
BOOL **ExecuteMacro** (BSTR strMacroName)
BOOL **RunFormScript** (BSTR strScriptName, CWLScriptWindowType mode)
BOOL **SendMail** (BSTR addr, BSTR strSubject, BSTR strText, BSTR attachments, BOOL bWithDialog)
void **SetAppBackgroundPic** (BSTR picture, int mode)
void **SetAppBackgroundColor** (short red, short green, short blue)
void **SetDefaultWinColor** (short red, short green, short blue)

Events

OnQuit (ICwlEventResult *bResult)
OnActivateApp (CWLApplicationNr AppNr)
OnWindowOpen (CWLApplicationNr AppNr, int windowId)
OnWindowClose (CWLApplicationNr AppNr, int windowId)
OnWindowActivate (CWLApplicationNr AppNr, int windowed)
OnScriptWindowMayClose (CWLApplicationNr AppNr, int windowId, ICwlEventResult *bResult)
OnPagePrinted (CWLApplicationNr AppNr, int windowId, int controlWinId, int controlId, int PageNr, BSTR formName)
OnPageStarted (CWLApplicationNr AppNr, int windowId, int controlWinId, int controlId, int PageNr, BSTR formName)

OnMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR MessageBoxText, ICwlEventResult *ButtonPressed)

OnBeforeMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR MessageBoxText, ICwlEventResult *ButtonPressed)

OnCompanyChange (BSTR CompanyNumber, int CompanyYear)

4.1.1. Properties

FullName [BSTR, read only]

Name of the EXE file with path, no extension,
e.g., "C:\WINLINE\cwlstart"

Name [BSTR, read only]

Name of the EXE file without path, no extension.
e.g., "cwlstart"

AppPath [BSTR, read only]

Path of the EXE file, backslash terminated.
e.g., "C:\WINLINE\"

WorkPath [BSTR, read only]

Current working folder of application, backslash terminated.
e.g., "C:\WINLINE\"

ServerPath [BSTR, read only]

Path to server folder, backslash terminated. Can be expressed as UNC path.
e.g., "C:\WINLINE\" or "\\SERVER\WINLINE\"

Visible [BOOL, read write]

Controls whether the entire window of CWL is visible or not.

TRUE	window is invisible
FALSE	window is visisble

CurrentModule [ICWLModule*, read only]

Pointer to the current module (see class ICWLModule).
There is always a current module.

CurrentWindow [ICWLWindow*, read only]

Pointer to the current window (see class CWLWindow).
There is always a current window, at least a Userform that belongs to the script (that may be invisible).

Application [ICwlLStart*, read only]

The application itself.

GlobalProperty (int PropertyNr) [BSTR, read write]

Sets or returns a user-specific property, which is identified with an number of the user. The property serves to transmit information between different scripts.

For example, one script sets a property:
GlobalProperty (1) = "A Test"
Another script (or the same one) reads the value in the property:
MsgBox GlobalProperty (1)

CurrentCompany [ICWLCompany*, read only]
Pointer to the current company (see Class 'CWLCompany').

Connection (VARIANT what) [ICWLDbConnection*, read only]

Creates a CWLDbConnection object dependant on the passed parameter. When the parameter is a string with 4 letters, it is assumed that this is a company number and the connection parameter of the company are read. When the value is a numerical value between 0 and 4 the connection parameters are configured as follows:
0 The current company
1 System database for data base connections, users, etc.
2 System data base for forms and windows
3 System data base for company-independent data
4 System data base for archive tables

4.1.2. Methods

Module(short nApplicationNr)

Parameter

NApplicationNr	Number of desired module (see CWLApplicationNr - Constants)

Return value (ICWLModule *)

Returns a pointer to the module with ID **nApplicationNr** (see also CWLApplicationNr - Constants) with type **ICWLModule**.
The respective application must not be active.

Quit

Ends the application.
Unsaved data are discarded!

ActivateModule(short nApplication)

Switches to the corresponding application with ID **nApplication** (see also CWLApplicationNr - Constants). See also MacroCommands. MApplication.

Parameter

nApplication	Number of desired module (see CWLApplicationNr - Constants)

Return value (VARIANT_BOOL)

TRUE	Application could be switched
FALSE	Invalid value for **nApplication**, or unable to switch to application module, e.g., when user has no authorization rights

ActivateExternalApp(short nApplicationId)

Starts the program marked as the "external program" number **nId**. The ID-numbering of the entered programs begins with 0. See also MacroCommands.MexternalApplication

Parameter

nApplicationId	Index of desired program (0 to 9)

Return value (VARIANT_BOOL)

TRUE	application could be started
FALSE	Unable to start application

ExecuteMacro(BSTR strMacroName)

Calls the specified macro in **strMacroName**. If this command is used in a script to call another macro, the called script is executed first and then the calling script proceeds. Only macros, not other scripts can be called with this command!
See also MacroCommands.MrunMacro

Parameter

strMacroName	name of the macro that should be started

Return value (VARIANT_BOOL)

TRUE	Macro could be started
FALSE	Unable to start macro

RunFormScript(BSTR strScriptName , CWLScriptWindowType mode)

Calls the system script contained in **strScriptName**. When this command is used in a script to call another script, the called system script is started and remains active. The calling script receives control again after the **FormDriver_OnActivate** event in the called system script (see also system script events).
The **mode** parameter specifies how the script will be started:
0 ... as standard window that will be hidden at module change
1 ... as modal window
2 ... as a window that remains over all other windows (and stays visible also at module change) See also MacroCommands.MrunForm

Parameter

strScriptName	Name of script to start
mode	Window type that should be used for the script to be started (see CWLScriptWindowType - Constants)

Return value (BOOL)

TRUE	Script could be started
FALSE	Unable to start script

SendMail(BSTR addr, BSTR strSubject, BSTR strText, BSTR attachments, BOOL bWithDialog)

Sends a mail on MAPI32 with the mail profile configured on the workstation.

Parameter

addr	Email address
strSubject	Reference text
strText	Mai lbody text
attachments	Attached document (with oath)
bWithDialog	TRUE: dialogue is shown FALSE: dialogue is not shown (an email address must be entered at least)

SetAppBackgroundPic (BSTR picture, int mode)

This method sets the background picture for the application window. This can be performed also in the Design tab area of menu item Parameters/Settings.
The **mode** parameter corresponds to the comboboxes shown there in "Display".

Parameter

picture	The picture name (either as file name with path, or with extension .FROMDB to use pictures from the database.
mode	Type of display: 1… centered 2… tiled 3… full screen

SetAppBackgroundColor (short red, short green, short blue)

This method sets the background color for the application window. The RGB parameters **red, green** and **blue** are the factors from which the color with a value from 0 to 255 will be set.

Parameter

red, green, blue	RGB color components. 0 to 255, where 0,0,0 is black and 255,255,255 is white.

void SetDefaultWinColor (short red, short green, short blue)

This method sets the standard background color for all program windows. The RGB parameters **red, green** and **blue** are the factors from which the color with a value from 0 to 255 will be set.

Parameter

red, green, blue	RGB color components. 0 to 255, where 0,0,0 is black and 255,255,255 is white.

4.1.3. Events

OnQuit(ICWlEventResult *bResult)

Fired before exiting the application.
If the exit should be prevented, you have to specify to the application with
bResult.Value = False

OnActivateApp(int AppNr)

Fired after switching to an application. The called application can be seen with the **AppNr** parameter (see CWLApplicationNr - Constants).

OnWindowOpen(int AppNr, int windowId)

Fired when a window with ID **windowId** in application with ID **AppNr** (see also CWLApplicationNr - Constants) is opened. The window is already present at this point in time. This event can be fired, however, at a point in time when the window is not yet displayed.

OnWindowClose(int AppNr, int windowId)

Fired when a window with ID **windowId** in application with ID **AppNr** (see also CWLApplicationNr - Constants) is closed. The window is at this point in time already unloaded and cannot be accessed!

OnWindowActivate (CWLApplicationNr AppNr, int windowId);

Fired when a window is activated (e.g., clicked with the mouse). For script windows this applies only when they are of the **cwlScriptWindowStandard** type (this is the window type for CTK windows when the window is defined as not modal). If a script is started with **CWLStart.RunFormScript**, the **mode** parameter must be set correspondingly.

OnScriptWindowMayClose (CWLApplicationNr AppNr, int windowId, ICwlEventResult *bResult);

Fired when a script window is closed (before the window is actually closed).
Closure of the window can be prevented with

bResult.Value = False

The **windowId** parameter is only set when the window is a standard CWL window or a script window of the **cwlScriptWindowStandard** type.

OnPageStarted (CWLApplicationNr AppNr, int windowId, int controlId, int PageNr, BSTR FormName)

Fired when a new page is started in printing. If output is to a preview, **controlId** is the ID of the preview. **windowId** is always the ID of the window that started the print out and not that of the preview window.
Usage example:
Before the print out of a page is started, you can change some of the variables which are to be printed (using the Vars property of the window which is printing).

OnPagePrinted (CWLApplicationNr AppNr, int windowId, int controlId, int PageNr, BSTR FormName)

Fired when a page is finished in printing. If output is to a preview, **controlId** is the ID of the preview. **windowId** is always the ID of the window that started the print out and not that of the preview window.

OnMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR MessageBoxText, ICwlEventResult *ButtonPressed)

Fired when a message box has been shown (after the user has closed the message box by pressing the confirm button). The actually pressed button in the message box can be overridden with

ButtonPressed.Value = x

(the buttons are numbered from left to right 1, 2, 3 .. etc.).

OnBeforeMessageBoxLaunched (CWLApplicationNr AppNr, int windowId, BSTR MessageBoxText, ICwlEventResult *ButtonPressed)

Fired before the message box is shown. The actually pressed button of the message box can be returned with

ButtonPressed.Value = x

This suppresses the message box display completely. (the buttons are numbered from left to right 1, 2, 3 .. etc). When -1 is returned, which is automatically the default value, the message box is displayed.

OnCompanyChange (BSTR CompanyNumber, int CompanyYear)

Fired when the company is changed (also when the fiscal year is changed in the application toolbar, i.e., this is effectively a company change).
The CompanyYear parameter passes the fiscal year in numerical format (for conversion in the text format that exists in the CWLCompany object conversion functions).

4.2. CWLScript

This inherent object represents the script itself.

Exposed in

- ☐ System Macros
- ☐ CTK Macros
- ☐ Lohn Macros
- ☐ Fakt Macros

CWLScript

Properties

BSTR **Name**
ICwlWindow ***CallingWindow**
ICwlWindow ***ScriptWindow**
long **ModalResult**

Methods

void **Stop** ()
void **Hide** ()
void **Show** ()

Events

OnScriptStart ()
OnScriptStop ()
OnParentClose ()

4.2.1. Properties

Name [BSTR, read only]

Name of script. See also MacroCommands.MName.

CallingWindow [ICwlWindow*, read only]

Only for CTK scripts.
Reference to the window object (see class CWLWindow) that is linked with the CTK script. The window that can display the script can be obtained with the ScriptWindow property.

ScriptWindow [ICwlWindow*, read only]

Returns the index to the script window itself. However, only when the script window is of the **cwlScriptWindowStandard** type - is always true for CTK windows (except when they were started modally) and is true for system scripts, only when they were started in the correct mode (see **RunFormScript**).

ModalResult [long, write only]

This property sets the result of the call of the modal window.

4.2.2. Methods

Stop
Unloads the script (no events are processed anymore). This method should not be mistaken for VBScript **Stop** command. Correspondingly the currently running script must always be ended with a next command **Exit Sub** or **Exit Function**.

Hide
Hides the window that is connected to the script.

Show
Displays the hidden script window again.

4.2.3. Events

OnScriptStart
Event is fired when the script is started.

OnScriptStop
Event is fired when the script is stopped.

OnParentClose
The event is fired in connection with CTK scripts when the window connected with the script is closed.

4.3. CWLCurrentModule

This object serves only to evaluated events for the currently active module. The object can only be used in connection with CTK scripts.

CWLCurrentModule

Properties

ICwlModule* **ActiveModule**

Events

OnActivate (int AppNr)
OnWindowOpen (int windowId)
OnWindowClose (int windowId)

Exposed in

☐ CTK Macros

4.3.1. Properties

ActiveModule [ICwlModule*, read only]

Returns a pointer to the currently active module.

4.3.2. Events

These events are only passed to the corresponding window script by CTK scripts. System scripts do not receive these events. Events are only passed to active CTK script windows in the current module.

OnActivate(int AppNr)

Fired when the module is opened.

OnWindowOpen (int windowId)

Fired when a window with ID **windowId** is opened.

OnWindowClose (int windowId)

Fired when a window with ID **windowId** is closed.

4.4. CWLCurrentWindow

This object only serves to evaluate events for the currently active window. The object can only be used with CTK scripts.

Exposed in

☐ CTK Macros

CWLCurrentWindow

Properties

ICwlModule* **ActiveWindow**

Events

OnActivate (int nWinId)
OnControlActivate (int nFgId)
OnCheck (int nFgId)
OnGridCheck (int nFgId)
OnGridChangeLine (int nFgId)
OnPushButton (int nFgId, ICwlEventResult *bResult)
OnCheckBox (int nFgId)
OnRadioButton (int nFgId)
OnChangeButton (int nFgId)
OnCheckUserfield (int nFgId, ICwlEventResult *bResult)
OnChangeFilter (BSTR FilterName, ICwlEventResult *bResult)
OnChangeCompanyYear (int CompanyYear, ICwlEventResult *bResult)
OnChangeCompany(const char *company, int CompanyYear, ICwlEventResult *bResult)
OnGridDblClick (int nFgId, ICwlEventResult *bResult)
OnDynamicMenuCommand (int nFgId, int MenuIndex, ICwlEventResult *bResult)

OnAfterEvent (int nFgId, int EventType, int Originalresult)
OnGridCheckUserColumn (int nFgId, int row, int column, ICwlEventResult *bResult)
OnSearch (int nFgId, ICwlEventResult *bResult)
OnGridSearch (int nFgId, int Zeile, int Spalte, ICwlEventResult *bResult))

4.4.1. Properties

ActiveWindow [ICwlWindow*, read only]
Returns a pointer to the currently active window.

4.4.2. Events

These events are only passed by CTK scripts to a corresponding window script. This means that elements in a window modified with CTK are defined as event triggers by the user (buttons, edit fields, etc.). The script linked to this window then receives the events.

void OnActivate(int nWinId)
Fires when the window is activated.

OnControlActivate(int nFgId)
Fired when an element with ID **nFgId** gets the focus.

OnCheck(int nFgId)
Fired when an edit field or combobox is exited, after the application has validated the content. If exit of the control is not allowed by the application (e.g., incorret input), the event is not fired.

OnGridCheck(int nFgId)
Fired when a cell with an edit field or combobox is left, after the application has validated the content. If exit of the cell is not allowed by the application (e.g., incorret input), the event is not fired.

OnGridChangeLine(int nFgId)
Fired when lines are changed in a grid.

OnPushButton(int nFgId, ICwlEventResult *bResult)
Event is fired when a button is pressed. The script receives the event before the application. When parameter '`bResult.Value = false' is optionally set, you can prevent the event from being passed on through to the application.`

OnCheckBox(int nFgId)
Fired when a checkbox is clicked.

OnRadioButton(int nFgId)
Fired when a radio button group is exited.

OnChangeButton(int nFgId)

Fired when a selection change is made within a radio button group.

OnCheckUserfield (int nFgId, ICwlEventResult *bResult)

Fired when an edit field or combo box inserted by a user in CTK is exited. You can prevent exit of the control field with parameter:
bResult.Value = False

After the event is fired, the entered value in the control is automatically copied to the associated window variable (assuming the bResult.Value = False is not set).
The current value in the entry field is obtained with property *ScreenContents* from the event. The original value is contained in property Contents (as well as the variables connected to the field).

OnChangeFilter (BSTR FilterName, ICwlEventResult *bResult)

Fired when the filter is changed in the Filter combo box in a window. The change of filter can be prevented with parameter:
bResult.Value = False

The FilterName parameter passes the name of the filter.

OnChangeCompanyYear (int CompanyYear, ICwlEventResult *bResult)

Fired when the currently set fiscal year is changed in the fiscal year combo box in the application tool bar.
The fiscal year change can be prevented with parameter:
bResult.Value = False

The CompanyYear parameter passes the fiscal year in numerical format (for conversion to a text format in the CWLCompany object conversion functions).

OnChangeCompany (const char *Company, int CompanyYear, ICwlEventResult *bResult)

Fired when the company is changed in the application (or the fiscal year). The company change can be prevented with parameter:
bResult.Value = False

The CompanyYear parameter passes the fiscal year in internal numerical format (for conversion in text format as it exists in the CWLCompany object conversion function. Normally, this event cannot be used since no window is allowed to be open during a company change (one exception is the Cockpit window).

OnGridDblClick (int nFgId, ICwlEventResult *bResult)

Fired when a double-click is made in a non-editable column of a screen grid (or the ENTER key is pressed).
The script receives the reporting call before the application and by setting parameter 'bResult.Value = false' you can prevent the application from executed the associated action with click/key press.

OnDynamicMenuCommand (int nFgId, int MenuIndex, ICwlEventResult *bResult)

Fired when a selection is made with a selection button in a window (e.g., printer/screen). This applies also when the button is selected with F5 shortcut. The selected menu item of the button is passed in MenuIndex, whereby the first menu item has index number 0 and the rest are numbered ascendingly.

The script receives the reporting call before the application and by setting parameter 'bResult.Value = false' you can prevent the application from executed the associated action with click/key press.

OnAfterEvent (int nFgId, int EventType, int Originalresult)

This event is fired after processing of certain events. This allows you to react to events in CWL after they have been executed (e.g., the pressing of a button)
This event is fired after the following event types:

- ☐ Push button (EventType = 7)
- ☐ Checkbox (EventType = 5)
- ☐ Radio button (EventType = 6)
- ☐ Radio button - Change-Event (EventType = 10)
- ☐ Listbox selection (EventType = 4)
- ☐ Double click in grid (EventType = 30)
- ☐ Checkbox in a grid (EventType = 26)

The *OriginalResult* - value contains the event result from CWL (normally this is 0, or as error result a value of -1, whereby the exact meaning is dependant on the particular usage precedent).

OnGridCheckUserColumn(int nFgId, int Row, int Column, ICwlEventResult* bResult)

Event is fired when a column field in a window grid that contains a combo box or edit field is exited in an inserted column. When the `bResult.Value = false` parameter is set, the program prevents the field from being exited.

OnSearch (int nFgId, ICwlEventResult *bResult)

This event is fired when the user clicks on the magnifying glass in an edit field or presses the F9 key.
In case the element was not created by the user, you can suppress the standard behavior by setting bResult.value = false (the Match Code is then not opened by the program).

OnGridSearch (int nFgId, int Zeile, int Spalte, ICwlEventResult *bResult)

This event is fired when the user clicks on the magnifying glass in an edit field in a grid column r presses the F9 key. This event is fired when the user clicks on the magnifying glass in an edit field or presses the F9 key.

4.5. CWLWindowVars

This object serves to provide access to window variables. The object access directly all variables that have been defined by the application within a window.

Exposed in

- ☐ System Macros
- ☐ CTK Macros
- ☐

CWLWindowVars

Properties
VARIANT **Value** (short nView, short nVar)
VARIANT **UserValue** (short nView, VARIANT Var)

Methods
BOOL **CreateVar** (short nVar, BSTR Type, int length, VARIANT Value)

- ☐

4.5.1. Properties

Value (short nView, short nVar) [VARIANT, read write]

Select the desired variable with **nView** and **nVar**. **nView** can either be the respective table number of the variable (e.g., company base info table T001 nView = 1) or 0. Variables with nView = 0 are created by the program and are not connected with any database table. **nVar** is the corresponding column number for database table variables (e.g., company base info street column C004 nVar = 4). If nView = 0, the number begins with 20. Numbers < 20 have the same meaning in all windows:

Nr	Meaning
11	Company number
12	Path
13	CompanyName
14	UserName
15	Version
16	Reporting date
17	Email address
18	Database version

Variables >= 20 depend on the respective window and correspond normally to the variables that are available in CWLPDFE.

UserValue (short nView, VARIANT Var) [VARIANT, read write]

Use this property to access to user-defind columns in a company table (the first user-defined column of a table: U000 can be accessed with UserValue (nView, 0) or userValue(nView, „U000") or UserValue (nView, „column name". The user-defined columns are always inserted from variable 500 (the example would function with Value(nView, 500).

With user-defined tables, the Uxxx column values are mapped to variables xxx. This allows for access by value as well as by UserValue with the same parameters (except when using the column name, which is only supported with the UserValue property).

Parameter

nView	Table number for which the variables are created
Var	Index of the UserVar (0 for U000), or the name of the column („U000") or the column description („name")

1.1.1. Methods

BOOL CreateVar (short nVar, BSTR Type, int length, VARIANT Value)

This function creates a new variable with a specific number in the range of user-defined variables (view 495). This variable is used as storage place for data in the elements of the window. When a variable already exists with the specified number, FALSE is returned.
FALSE is also returned when the specified value in **Value** cannot be converted to a variable data type (e.g., an invalid date).

The number of the variable to be created is specified with **nVar**.
The number can begin with 0 and contain a maximum of 1000 entries. The numbers do not have to be consecutively numbered in ascending order. The type of the variable is determined with **Type,** whereby the following types are possible:

Type	

1	Text variable (length variable)
2	Integer
4	Double
6	Date with time

The length of the variable is specifed with **length**. 0 can be specified for all other data types.
A value can be passed with **Value** that is entered as the initial value of the variable. This parameter is optional.

4.6. CWLEventResult

This object serves to return values for events which demand a result. Due to the nature of VBScript, the value cannot be set directly, but must rather use the syntax: result.Value = true (or false).

Exposed in

☐ System Macros
☐ CTK Macros

4.6.1. Properties

Value [BOOL, read write]

This property stores the result of the event.

Example:

```
Sub CWLStart_OnScriptWindowMayClose(AppNr, windowId, bResult)
        If <Condition> Then
                bResult.Value = True
        Else
                bResult.Value = False
        End If
End Sub
```

4.7. CWLSearchResult

This object serves to access the result of an SQL query that can be executed in the CWLCompany object. Depending on the query, the column of the result, the name of the column and the number are available.

Exposed in

☐ System Macros
☐ CTK Macros

VARIANT **Value** (VARIANT ColumnIndexOrName)
BSTR **ColumnName** (short nColumnIndex)
Methods
BOOL NextRecord ()
Close ()

4.7.1. Properties

Value(VARIANT IndexOrName) [VARIANT, read only]

Returns the result of the query with a corresponding index (or column name).

MaxColumnIndex [short, read only]

Returns the index of the last result column (corresponds to number of result columns - 1).

ColumnName (short Index) [BSTR, read only]

Returns the name of the column with the **Index** number, whereby the index can be between 0 and **MaxColumnIndex**.

Example

```
Dim text, result, i
edtResult = ""  ' Text field for result

On Error Resume Next

' Search in table T024 for product ,10001`
' from version 8.0 the company and fiscal year must be defined in the query as well!
' the current company is represented with placeholder '~~~~' and the current fiscal year with yyyy
'the result is a CWLSearchResult - object whose default property is
' MaxColumnIndex, which = -1 when no record was found
Set result = CWLStart.CurrentCompany.SearchRecord ("T024", "C002 = '10001' and MESOCOMP = `~~~~' and MESOYEAR = yyyy")
If result < 0  Then
        If err <> 0 Then
                ' error resulted (e.g. C002 is not a valid column)
                MsgBox err.description
                Exit Sub
        Else
                ' Product not found
                MsgBox "Could not find the requested record."
                Exit Sub
        End If
End If

' all columns and displayed with column name and contents
' in a text field
i = 0
For i =0 To result.MaxColumnIndex
        text = result.ColumnName (i) & ": " & result.Value (i) & chr (13)
        If err <> 0 Then
                MsgBox err.description
                Exit Sub
        End If
        edtResult = edtResult + text
Next

' Result of the routine in text field edtResult
' C002: 10001
' C003: Bike 26"
```

```
' C011: 10001
' C014: 0
' C020: 0
' etc.
```

4.7.2. Methods

NextRecord()
Result (BOOL)

Returns TRUE when a further data records has been read, or FALSE when no more data records are left.

Close
Closes the object and discards all data saved therein.

4.8. GeneralScriptFuncs

This object opens windows of the operating system (or the VBScript) on the client PC, which in the case of the EWL are opened on the server.
MsgBox and *InputBox* are functions that can be used with the VBScript Engine (the functions can be used in the CWL without this object as VBScript functions when compatability with the EWL is not required).
FileDialog opens the file selection window of the operating system.

Available in
☐ eveywhere

GeneralScriptFuncs
Properties
BOOL WaitCursor
Methods
int **MsgBox** (BSTR prompt, VARIANT buttons, VARIANT title)
BSTR **InputBox** (BSTR prompt, VARIANT default, VARIANT title)
BSTR **FileDialog** (BOOL Open, BSTR Extension, BSTR Filename, VARIANT Filter)

4.8.1. Properties

WaitCursor [BOOL, read write]
Set this property to display a timer „sand glass" icon as mouse cursor.

4.8.2. Methods

int MsgBox (BSTR prompt, VARIANT buttons, VARIANT title)
This function corresponds to the VBScript function "MsgBox".

BSTR InputBox (BSTR prompt, VARIANT default, VARIANT title)
This function corresponds to the VBScript function "InputBox", the parameters default and title are

exchanged, however.

BSTR FileDialog (BOOL Open, BSTR Extension, BSTR Filename, VARIANT Filter)

Use this function to open the file selection dialogue in the operating system.

When parameter **Open** is set to TRUE, the "Open File" dialogue window is displayed, otherwise the "Save File" dialogue.
Use the **Extension** parameter to specify a file extension when the user does not enter an extension.
Use the **Filename** parameter to specify a file name including pacth that is preset when beginning the entry.
Use the **Filter** parametr (optional) to fill the file type selection box with appropriate data types.
When no parameter is passed, option "All files (*.*)" is used. The separate file types are separated with a '|' character, the end is denoted with two '||' ("Text files (*.txt)|*.txt|Spool files (*.spl)|*.spl||").

When a file is selected, the selected file name including patch is returned by the function. When cancelled, an empty string is returned.

Examples:

general.msgbox general.FileDialog (True, "txt", "c:\temp\scripttest*.txt", "Text files (*.txt)|*.txt|Spool files (*.spl)|*.spl||")

4.9. CWLTable

This object represents an opened database table. It allows the access of values from the table, the insertion of new values to the table, updates to the table and deletion of records from the table. This basic functionality operates on the data record level (one data record is changed at a time that is located by a unique key).

Each column in the table is created as a variable in the associated window, which allows for use of the table contents directly in a window or PDF.

Additionally a SQL expression can be run of a table. The values returned are written to the table variables, which in turn are available for use in the window or PDF.

Available in

☐ System scripts
☐ CTK window scripts

CWLTable
Properties
BSTR Name
BOOL Valid
int MaxColIndex
Methods
VARIANT **Value** (VARIANT column)
void **Value** (VARIANT column, VARIANT newValue)
BOOL **Get** (BSTR Key, VARIANT ExpandKey)
BOOL **Update** ()
BOOL **Insert** ()
BOOL **Delete** (BSTR Key, VARIANT WhereStmt)
void **CopyToWindow** (short Window)
CWLSearchResult ***Select** (BSTR SelectStmt)

4.9.1. Properties

Name (BOOL, read only)

The name of the table is held in this property.

Valid (BOOL, read only)

This property contains the information whether the table was successfully opened.

MaxColIndex (int, read only)

The index of the last defined column is saved in this property (this corresponds to the last of the variables that were created for this table).

4.9.2. Methods

VARIANT Value (VARIANT column)

Use this method to access the variables in the table (the table column values).

Parameter

Column	The column index or column name. For user-defined tables, the column name can also be used. (the column name is language-specific, howver, which could lead to problems when the table is being used in multiple program languages)

Return value: the column value

void Value (VARIANT column, VARIANT newValue)

Use this method to change a column value.
Parameter

Column	The column index or column name. For user-defined tables, the column name can also be used. (the column name is language-specific, howver, which could lead to problems when the table is being used in multiple program languages)
newValue	The new value for the variable (column)

Example

```
if table.value(1) > 100 then
    table.value(1) = table.value(1)/100
end if
```

BOOL Get (BSTR Key, VARIANT ExpandKey)

Use this method to read out a data record with a unique 'Key'.
When the data record is found, the grid variables (columns) are filled with the contents of the data record.

Parameter

Key	The unique key that is defined in the specified column for OpenTable
ExpandKey	The key must include the company code and fiscal year for mesonic database tables to ensure that the record is unique. When this parameter is not specified, the value is set based on the table type.

Return value: When the record is found, TRUE is returned, otherwise FALSE.

BOOL Update ()

Use this method to update the data record last loaded. When the table variables still contain valid values, the data record must not be loaded with Get.

Return value: TRUE when update of the record was successful

BOOL **Insert** ()
Use this method to insert a new data record. The table variables must be filled with the new values in advance.

Return value: TRUE when insertion of the data record was successful

BOOL **Delete** (BSTR Key, VARIANT WhereStmt)
Use this method to delete one or more data records.

Parameter

Key	The unique key that is defined in the specified column for OpenTable
WhereStmt	When a WhereStmt is passed, all data records are deleted that correspond to the WhereStmt. An empty string must be passed in the key.

Return value: TRUE when successful, otherwise FALSE.

void CopyToWindow (short Window)

Use this method to copy the table variables into the variables of view 495 (i..e, the variables that are defined in MDP projects as user-defined variables). When the method is called, existing variables are overwritten or changed so that they correspond with the types of the column values.

Parameter

Window	Window in which the variables will be changed.

CWLSearchResult *Select (BSTR SelectStmt)

Use this method to execute a SELECT statement on a table. The column values of the returned data records are copied into the table variables of the table. The values can also be read out with methods of the

CWLSearchResult object.
The function replaces the ~~~~ with the current company (text column) and yyyy with the current fiscal year (numeric) in parameter **SelectStmt**.

Parameter

SelectStmt	The WHERE expresssion that is run in a SELECT * on the table.

Return Value: a CWLSearchResult object

Example

```
Set tDW = conn.OpenTable2 (699, 900) ' open user-defined table
' sort all data records by column U000
Dim search
Set search = tDW.Select("order by U000")
If search.RowCount > 0 Then
    Do
        ' get employee record from tDW
        T401.get tDW.value(0)
        grid.AddLine
        If search.NextRecord = False Then  ' was the last record?
            Exit Do
        End If
    Loop
End If
```

5. Classes

The following classes are contained in the CWL object model, from which objects are derived from the Root Object **CWLStart**.

5.1. CWLCompany

This object represents the currently loaded company. The current company data can be read with it. Additionally, any desired SQL Select query can be executed to read from any existing company table.

Exposed in

☐ System Macros
☐ CTK Macros
☐

CWLCompany

Properties

BSTR **Nr**
BSTR **Name**
ICWLSearchResult ***SearchResult**
BOOL **Valid**
VARIANT **Value** (short nVar)
CWLDbConnection ***Connection**
Int **CompanyYear**

Methods

ICWLSearchResult ***SearchRecord** (BSTR strTableName, BSTR strWhereStatement)
long **UpdateRecord** (BSTR strTableName, BSTR strUpdateStatement, BSTR strWhereStatement)
void **Refresh** ()
CWLDbConnection **GetSystemConnection** (CWLSystemServerType what)
BSTR **ConvertCompanyYearToString** (int YearValue)
int **ConvertCompanyYearStringToValue** (BSTR YearString)

Events

OnUpdateTable (short TableNum)
OnInsertTable (short TableNum)
OnDeleteTable (short TableNum, BSTR Key, BSTR WhereStmt)

5.1.1. Properties

Nr [BSTR, read only]
Returns the company number.

Name [BSTR, read only]
Returns the company name.

Valid [BOOL, read only]
Returns TRUE when the current company is loaded.

Value (short nVar) [VARIANT, read only]

Returns the value of the **varNo** column of the currently loaded company. Columns which do not exist are returned with variable type "Nothing" (not all company columns are continuously numbered). If an invalid variable number is passed, a Runtime Error will result.

SearchResult [ICWLSearchResult *, read only]

Returns the Result Object that contains the result value (see CWLSearchResult - Object).

Connection [CWLDbConnection *, read only]

Returns as result the CWLDbConnection object of the current company.

CompanyYear [int, read only]

Returns as result the current fiscal year (in internal numerical format) of the fiscal year that is currently set i the fiscal year list box. The value can be converted to a text format with function **ConvertCompanyYearStringToValue**.

5.1.2. Methods

Refresh

Company data in the program are refreshed from the database.

SearchRecord (BSTR strTableName, BSTR strWhereStatement)

You can search for a data record according to the **strWhereStatement** parameter in the **strTableName** table. If the data record is found, the individual results can be read with the returned CWLSearchResult Object or the **SearchResult** property can be evaluated.

In place of the company in the table name you can specify ~~~~ . These four characters are then replaced by the currently loaded company (e.g., for company 300M, T024~~~~ is understood as T024300M). The same convention can be used for the current fiscal year with placeholder yyyy (e.g., where mesoyear=yyyy).

strWhereStatement contains the condition which data record should be searched for (e.g., 'C002' = '10001'). From version 8.0 the query statement must include the current company and fiscal year!

The first data record found is always made available in **SearchResult.** When the result consists of several data records together, the results of the other data records are not loaded.

If the query syntax is faulty or if you attempt to query a non-existent table, you will receive a Runtime Error. If a data record is not found, **SearchResult** contains as MaxColumnIndex -1. MaxColumnIndex is also the standard property of **SearchResult**, which means that the query can also be executed as follows:

```
Set result = CWLStart.CurrentCompany.SearchRecord (…)
if result = -1 then ' result is the same as result.MaxColumnIndex
        MsgBox "Data record not found"
end if
```

Parameter

strTableName	Name of table
strWhereStatement	Query argument for the sql

	statement. The company and fiscal year must always be specified (MESOCOMP = '~~~~' and MESOYEAR = yyyy)

Return value (ICWLSearchResult *)

UpdateRecord (BSTR strTableName, BSTR strUpdateStatement, BSTR strWhereStatement)

Columns in table **strTableName** will be updated with values that are passed with statement **strUpdateStatement** for all data records corresponding to statement **strWhereStatement**. (e.g., UPDATE *strTableName* SET *strUpdateStatement* WHERE *strWhereStatement* ➔ UPDATE *T024* SET *C003 = 'Herren Rennsportrad'* WHERE *C002 = '10005' and MESOCOMP =* '~~~~' and MESOYEAR = yyyy).

When no values can be updated (the **strWhereStatement** matches no existing data record), a value of FALSE is returned.
When the query is syntactically not valid or a non-existant table is specified, a runtime error occurs.

Parameter

strTableName	Name of table
strUpdateStatement	Defintion of values for the columns that should be updated (e.g., "C001 = 3, C002 = 'Text'")
strWhereStatement	Conditional statement for query (valid SQL expression, as it must be formulated in a where statement in a SQL expression) The company and fiscal year must always be selected (MESOCOMP = '~~~~' and MESOYEAR = yyyy)

Return value (long)

> 0	Number of successfully updated data records.
0	The update was not performed.

CWLDbConnection GetSystemConnection (CWLSystemServerType what)

Returns the CWLDbConnection object foir the desired system database (compare with the CWLSystemServerType Constants in the Appendix).

BSTR ConvertCompanyYearToString (int YearValue)

This method converts the internal numerical format of the fiscal year to a text format as it is displayed in the fiscal year list box.

Int ConvertCompanyYearStringToValue (BSTR YearString)

This method converts the fiscal year in text format as it is displayed in the fiscal year list box to the internal numerical format.

5.1.3. Events

OnUpdateTable (short TableNum)

This event is triggered when an update is made on a mesonic database table that has been appended with user-defined columns.
The developer can modify and work with the data to be written to the database before the actual table update.

Parameter

TableNum	Table that has been expanded with user-defined table columns

OnInsertTable (short TableNum)

This event is triggered when an insert is made on a mesonic database table that has been appended with user-defined columns.
The developer can modify and work with the data to be written to the database before the actual table update.

Parameter

TableNum	Table that has been expanded with user-defined table columns

OnDeleteTable (short TableNum, BSTR Key, BSTR WhereStmt)

This event is triggered when a delete is made on a mesonic database table that has been appended with user-defined table columns.
The developer can perform their own operations depending in on the type of data records to be deleted.

Parameter

TableNum	Table that has been expanded with user-defined table columns
Key	Key column of the data record to be deleted. When the value is empty, a *WhereStmt* is used for the deletion.
WhereStmt	A 'where' statement that selects the data record to be deleted. The expression is only used when the *Key* is empty. When both are empty, all data records of the current company are deleted.

5.2. CWLDbConnection

This object describes the database connection. The type of database, name of database and name of SQL server are listed.

Exposed in

☐ System Macros
☐ CTK Macros
☐

CWLDbConnection
Properties
CWLDbConnectionType **Type**
BSTR **DatabaseName**
BSTR **ServerName**
Methods
CWLSearchResult ***Select** (BSTR Statement);
CWLTable* **OpenTable** (BSTR strTableName, int ViewNumber, BSTR KeyColumn, int WindowId, VARIANT UseCompany);
void **CloseTable** (CWLTable* pTable);
BOOL **ExecuteSQL** (BSTR Statement);
CWLTable* **OpenTable2** (short Number, short WindowId, VARIANT KeyColumn);

5.2.1. Properties

Type [CWLDbConnectionType, read only]
Returns the database type (compare with CWLDbConnectionType - Constants in the Appendix).

DatabaseName [BSTR, read only]
Returns the name of the database.

ServerName [BSTR, read only]
Returns the name of the SQL server.

1.1.2. Methods

Select (BSTR Statement)
A SQL Statement is executed on the database connection. The statement must be a SELECT statement that contains the (NOLOCK) attribute!

Parameter

Statement	SQL expression (must be a SELECT statement)

Return value: a CWLSearchResult - object with results of the query

Example

```
Dim conn, result
'Database connection  of current company
Set conn = CWLStart.CurrentCompany.Connection

' obtain the current company (with current FY)
Set result = conn.Select ("Select * from t001 (NOLOCK) where mesocomp = '~~~~'
And mesoyear = yyyy")

' Output the current company name
general.MsgBox result.value("c000")
```

CWLTable OpenTable (BSTR strTableName, int ViewNumber, BSTR KeyColumn, int WindowId, VARIANT UseCompany);

This method opens the table of specifed name within the current database connection.
This method is used with tables that do not conform to the mesonc table naming convention (Txxx).

Parameter

strTableName	Name of Table
ViewNumber	Table number with which the colu mn value was created as varible. The variables are created in the window with the specified WindowID and may not be used by any other opened table.
KeyColumn	The column in the table that is used in the CWLTable methods, which use a key as parameter (e.g., get)
WindowId	The program window for whose variables the variables for the columns of the table were created (can then be used with CwlWindow.Vars(ViewNumber,x))
UseCompany	In case a mesonic standard table is opened with this method, you can specify with this optional parameter whether the used key should automatically be extended with a get/delete (when a Txxx tables is used, the program sets the parameter automatically to TRUE. When a key column has been specified that does not correspond to the default key of the table, the parameter must be specified with FALSE, otherwise an incorrect key will be created.

Return value: a CWLTable object

Example

```
On Error Resume Next
Dim conn, table
'Database connection of the current company
Set table = conn.OpenTable ("MyTable", 497, "No", 900)
```

```
If table Is Nothing Then
      msgbox "Table 'MyTable' was not found"
End If
If Not table.get ("1") Then
      msgbox "Get from 'MyTable': the data record was not found!"
Else
      msgbox "Get from 'MyTable (1)': " & table.value(1)
End If
```

CloseTable (CWLTable Table)
Closes the opened table and discards the created variables.

Parameter

Table	The CWLTable object that was opened with OpenTable

BOOL ExecuteSQL (BSTR Statement)
This method executes any SQL expression. Select statements are executed, but since the results of the select are not returned, this is not the actual purpose of this method.

Parameter

Statement	SQL expression

Return: FALSE is returned with an error, otherwise TRUE.

CWLTable OpenTable2 (short Number, short WindowId, VARIANT KeyColumn)
This method opens the table numbered 'Number', whereby the table must be named according to the mesonic naming convention (Txxx; xxx is a number with leading zeros), i.e., it can be used alternatively to the OpenTable method, when the table is a mesonic standard table in the WinLine database.

This method corresponds to the OpentTable method, but is can only be used upon tables named „Txxx".

Parameter

Number	Table number (xxx in Table name Txxx)
WindowId	Window in which the variables for the table columns have been created.
KeyColumn	Columns that serve as keys with get/update or delete calls. When the parameter is not specified, the default key column is used.

Return: a CWLTable Object

5.3. CWLModule

Objects of this class represent CWL modules such as START, ACC1, ACC2, etc. - see also CWLApplicationNr - Constants.

Exposed in

☐ System Macros

☐ CTK Macros

☐

CWLModule
Properties
ICwlWindow* **CurrentWindow**
ICwlWinCollection* **Windows**
BSTR **Name**
Methods
BOOL **Activate** ()
BOOL **IsWindowOpen** (short WinId)

☐

5.3.1. Properties

CurrentWindow [ICwlWindow*, read only]

Returns a pointer to the current window in this module. If no window is open, nothing is returned. A Runtime Error occurs when an unexpected problem crops up in the function.

A script window of the **cwlScriptWindowStandard** type is also returned as current window. All other script windows do not fall into this category and are not recognized as CurrentWindow.

Example 1 (tests for Runtime Error with missing window)

```
On Error Resume Next
err.clear
myname = cwlstart.CurrentModule.CurrentWindow.Name
If err <> 0 Then
  myname = "No window is active:" & err.number
End If
On Error Goto 0
MsgBox myname
```

Example 2 (tests for non-existant window)

```
Set curwin = CurrentModule.CurrentWindow
If TypeName(curwin) = "Nothing" Then
        msgbox "No window is active"
Else
        msgbox curwin.Name
End If
```

Windows [ICwlWinCollection*, read only]

Returns a pointer to an object of the **CWLWinCollection** class, with which you can access all active windows of a module.
Windows must be loaded, not necessarily visible, to be able to be accessed with this Collection.

Name [BSTR, read only]

Name of module.

5.3.2. Methods

Activate

Activates a corresponding module. The module has to have been activated before (i.e., it must already be 'present'). Opposite to **MacroCommands**, **MApplication**(ApplicationNr) does not start a module, but merely switches over to an already started module.

Return value (VARIANT_BOOL)

TRUE	Module could be started
FALSE	Module could not be started (e.g., with missing authorization rights)

BOOL IsWindowOpen (short WinId)

This method tests whether the window with the specified number is open.

Parameter

WinId	Window number

Return: TRUE when window is opened, otherwise FALSE

Example

```
'when the module has not been activated, it is not possible
'to access the module
if TypeName (cwlstart.Module(cwlFAKT)) = "Nothing" then
        exit sub
endif
myresult = cwlstart.Module(cwlFAKT).activate
```

5.3.3. Usage

A pointer to an object of class **CWLModule** can be obtained in the following manner:

Current Module

```
myModule = CWLStart.CurrentModule
```

Any Module

```
myModule = CWLStart.Module(cwlFAKT)
```

5.4. CWLWinCollection

Offers access to all objects of the **CWLWindow** class.

Exposed in

- ☐ System Macros
- ☐ CTK Macros

CWLWinCollection
Properties
long **Count**
ICwlWindow* **Item** (long nWinId)
ICwlWindow* **NamedItem** (BSTR strWinName)
ICwlWindow* **IndexedItem** (int nIndex)
Methods
BOOL **Add** (long nWinId)

5.4.1. Properties

Count [long, read only]

Number of objects in this collection.

5.4.2. Methods

Item (long nWinId)

Returns a pointer to a window with the specified ID **nWinId**.
The ID corresponds to that in CTK.

If the window with the specified ID does not exist, the return value contains nothing (must be tested with Typename (var) = "Nothing").

Parameter

nWinId	Window number of desired window

Return value (ICwlWindow*)
Pointer to window

NamedItem(BSTR strWinName)

Returns a pointer to a window with specified name **strWinName**.
The name corresponds to the Title property of the window in CTK.
If the window with the specified ID does not exist, the return value contains nothing (must be tested with Typename (var) = "Nothing").

Parameter

strWinName	Window title of the desired window

Return value (ICwlWindow*)
Pointer to the window

IndexedItem(int nIndex)

Returns a pointer to a window in the Collection with Index **nIndex**, beginning with 0.
If the window with the specified ID does not exist, the return value contains nothing (must be tested with Typename (var) = "Nothing").

Parameter

nIndex	Index of the desired window (index of all open windows, beginning with 0)

Return value (ICwlWindow*)
Pointer to the window

Add(long nWinId)

Opens a window in this module with window ID **nWinId**. The command corresponds to the macro command **MacroCommands.MWindow**.
If the window cannot be opened, FALSE is returned.

Parameter

nWinId	Number of the desired window

5.4.3. Usage

An object of the **CWLWinCollection** class exists only in the **Windows** property of a module (**CWLModule**). This collection always contains all loaded windows (which can be visible or invisible) and permits access to the same, e.g., access to **Product – Base Info** Window:

```
myWindow = CWLStart.Module(cwlFAKT).Windows.Item(210)
```

5.5. CwlWindow

This class defines window objects in the **CWLWinCollection** collection.

Exposed in

☐ System Macros
☐ CTK Macros

CWLWindow
Properties
Short **CurrentField**
BOOL **Visible**
Short **Id**
ICwlWindowVars* **Vars**
BSTR **Name**
CWLWindowTypes **Type**
ICwlFgCollection* **Controls**
ICwlFgControl* **CurrentControl**
BSTR **CurrentFilter**
Int **CurrentCompanyYear**
Methods
long **Close** ()
void **Activate** ()
void **Refresh** ()

5.5.1. Properties

CurrentField [short, read write]

ID of the field (Element) which has the focus in this window.
The focus shift simulates exiting the current field and checks whether the field could be exited. Only when this functions (i.e., application logic permits), is the focus shifted to the desired field.
If the ID 0 is used, the focus is set to the next field in the TAB order. If the focus cannot be set, a Runtime Error is produced.

Visible [BOOL, read write]

Determines whether a window is visible or not.
This property can be set for UserForms, but is read-only for CWL system windows.

Id [short, read only]

Unique ID of the window. Corresponds to the ID in CTK.
The ID of UserForms and preview windows is dynamically assigned and depends on the number of opened windows.

Vars [ICwlWindowVars*, read only]

Gives access to the variables used in this window.

Name [BSTR, read only]
Name of window.

Type [CWLWindowTypes, read only]
Type of window. See also Constants - CWLWindowTypes.

Controls [ICwlFgCollection*, read only]
Contains a collection of all elements of the **CWLFgControl** window class.

CurrentControl [ICwlFgControl*, read only]
Pointer to the field (element) that has the focus in this window.

CurrentFilter [BSTR, read only]
The name of the currently active filter (only when the window contains a filter combobox in the window toolbar).

CurrentCompanyYear [int, read only]
The current fiscal year in internal numerical format. The format can be transformed with a function fromt he CWLCompany object to text format and vice versa ConvertCompanyYearToString and ConvertCompanyYearStringToValue).

5.5.2. Methods

Close
Closes a window.
Can only be used with CWL standard windows and simulates pressing the EXIT button in the window.
If you attempt to close a window of another type, the function returns 0. If window closing was successful, 1 is returned.

Refresh
All fields in the window are refreshed. Any changes to variables that are displayed in fields in the window are thus made visible.

5.6. CwlFgCollection

Offers access to a collection of objects in the **CWLFgControl** class.

Exposed in
☐ System Macros
☐ CTK Macros

CwlFgCollection
Properties
long **Count**
ICwlFgControl* **Item** (long nFgId)

ICwlFgControl* **IndexedItem** (long nIndex)

5.6.1. Properties

Count [long, read only]
Number of objects in this collection.

5.6.2. Methods

Item(long nFgId)
Returns an object of the **CWLFgControl** class from this collection. Specify the ID of the element with **nFgId**. This ID corresponds to the ID of the corresponding element in the window displayed in the CTK program.

Parameter

nFgId	Number of the desired Fg control

Return value (ICwlFgControl*)
Pointer to the element

IndexedItem(long nIndex)
Returns an object of the **CWLFgControl** class from this collection. **nIndex** corresponds to the position of the element in the collection (0..**Count**).

Parameter

nIndex	The index of the desired Fg control in the list of existing controls, beginning with 0

Return value (ICwlFgControl*)
Pointer to the element

5.6.3. Usage

Each object of the **CWLWindow** class contains a property **Controls** of the **CWLFgCollection** class. This permits you to access all elements of a window (edit fields, texts, etc.). The element ID can be seen in the window view in the CTK program.

Example for accessing **Product number** (ID=101) field in the **Product Base Info** (ID=245) window in the **ACC2** (ID=cwlFAKT) module:

```
Set myElement = CWLStart.Module(cwlFAKT).windows.item(245).controls.item(101)
```

5.7. CwlFgControl

A object of this class represents an element in a window. This can be an edit field, a label or any other element in the window. See also CWLControlTypes - Constants.

Exposed in

☐ System Macros
☐ CTK Macros

CWLFgControl

Properties

short **Id**
VARIANT **Contents**
BSTR **Text**
long **View**
long **Var**
long **Line**
long **Column**
CWLControlTypes **Type**
long **Font**
long **Height**
long **Width**
ICwlPreview* **Preview**
ICwlSpreadSheet* **SpreadSheet**
VARIANT ScreenContents
VARIANT GridRedraw
ICWLGrid* Grid
BOOL Active

Methods

long **GridLines** ()
long **PushButton** (VARIANT PostIt)
long **TreeExpand** (BOOL bAll)
long **TreeCollapse** (BOOL bAll)
long **TreeSelect** (BSTR strSearch, BOOL bSearchExact)
long **ListboxSelect** (long nItemIndex)
BOOL **SetCurrentGridCell** (long Row, long logColumn)
BOOL **GetCurrentGridCell** (VARIANT * Row, VARIANT * logColumn)
VARIANT **GetGridCellValue** (long Row, long logColumn)
void **Validate** ()
BOOL **SetGridColReadOnly** (long logColumn, VARIANT bSet)
BOOL **GetGridColReadOnly** (long logColumn)

5.7.1. Properties

Id [short, read only]
Element ID, corresponds to the ID the window in the CTK program.

Contents [VARIANT, read write]

Content of the element, independent of type. See also CWLControlTypes - Constants and the **Type** property. In the case of a gird, the value is the current value in the grid cell.
When setting this parameter, the element must be the current element. Setting the element calls implicitly the **Validate** function, in order to permit the application the opportunity to process the element value. The focus is then set to the next element.
Static fields do not have to be the current element in the window, since this is not supported. The **Validate** function is not automatically started for these fields.
With special data types (e.g., dates) the value that will be set in the field should be converted to a corresponding type, otherwise only an internal conversion is executed, which may not lead to correct values in some cases. If the date is passed as text, its format must be DD.MM.YYYY HH:MM:SS (the clock time value is optional).

Text [BSTR, read only]
Corresponds to the **Title** property of the element in CTK.

View [long, read only]
Assigned program variable, View (table number)

Var [long, read only]
Assigned program variable, Var (column number in table)

Line [long, read only]
Corresponds to the **Row** property of the element in CTK.

Column [long, read only]
Corresponds to the **Column** property of the element in CTK.

Type [CWLControlTypes, read only]
Element type, see also CWLControlTypes - Constants.

Font [long, read only]
ID of font used for this element. Corresponds to the font ID in CTK.

Height [long, read only]
ID of font used for this element. Corresponds to the font ID in CTK.

Width [long, read only]
Corresponds to the **Width** property of the element in CTK.

Preview [ICwlPreview*, read only]
Pointer to an object of the **CWLPreview** class when this element is a Preview element (Type=cwlControlPreview) - see also CWLControlTypes - Constants and **CWLPreview**.

ScreenContents [VARIANT, read only]

This property returns the current value of the entry field (without validation or adjustment by the program). The **Contents** property returns the contents of the variable that is connnected with the field. This means that during OnCheck events only the **ScreenContents** property contains the currently entered value (on the screen), whereby the variable is filled after the event. The **Contents** property is in this sense contains the „original" value during the OnCheck event.
Outside of the OnCheck event, both properties should always contain the same value.

GridRedraw [VARIANT]

When many changes are carried out in a grid control, processing can become very slow when each individual change is displayed directly on the screen display. Using this property, you can turn off the grid screen refresh while the changes are being made. When the property is set then to TRUE, all the changes are displayed at once on the screen.

SpreadSheet [ICwlSpreadSheet*, read only]

Pointer to an object of the **CWLSpreadSheet** class when this element is a SpreadSheet element (Type=cwlControlSpreadsheet) - see also CWLControlTypes - Constants and **CWLSpreadSheet**.

5.7.2. Methods

PushButton (VARIANT PostIt)

If the element is a PushButton (Type= cwlControlButton), you can fire the Push event with this method.
The element must be the current element in the window, otherwise a runtime error will be generated.
The parameter is optional and is passes as FALSE when not otherwise specified. This executes the method directly. When TRUE is passed, the button is first pressed when the current VB script function is ended. This allows you to close the current window in the VB script attached to the window. Were the push button to be immdediately executed, a system error would be produced, since the VB script would find no object after returning from the push button execution, i.e., the window would have been closed and thus the connection with the VB script ended.

Parameter (optional)

FALSE	(default) the button push action is directly (immediately) executed
TRUE	(default) the button push action is executed after th VB script is ended.

Return value (long)
Contains an application-specific value, the default value is 0, but it can assume other values depending on the window.

TreeExpand (BOOL bAll)

If the element is a Tree control (Type= cwlControlTree), you can open the currently selected node of the tree with this method.
Depending on the **bAll** parameter, you can also expand the entire tree.
The element must be the current element in the window, otherwise a runtime error will be generated.

Parameter

TRUE	Expands the entire tree
FALSE	Expands only the selected node for a level

Return value (long)

Contains an application-specific value, the default value is 0, but it can assume other values depending on the window.

TreeCollapse(VARIANT_BOOL bAll)

If the element is a Tree control (Type= cwlControlTree), you can close the currently selected node of the tree with this method.
Depending on the **bAll** parameter, you can also close the entire tree.
The element must be the current element in the window, otherwise a runtime error will be generated.

Parameter

TRUE	Closes the entire tree
FALSE	Closes only the selected node

Return value (long)
Contains an application-specific value, the default value is 0, but it can assume other values depending on the window.

TreeSelect(BSTR strSearch, BOOL bSearchExact)

If the element is a Tree control (Type= cwlControlTree), you can search for a tree element with this method. Depending on the **bSearchExact** parameter, the element is found during an exact search, when the complete text is passed, otherwise it is sufficient, when the tree text begins with the search text.
The element must be the current element in the window, otherwise a runtime error will be generated.

Parameter

strSearch	Search text for the search in tree menu
bSearchExact	Search result must match exactly (TRUE) or the search result must begin with strSearch value (FALSE)

Return value (long)
Contains an application-specific value, the default value is 0, but it can assume other values depending on the window.

GridLines

If the element is a grid (Type=cwlControlGrid), this method returns the number of accessible lines in this grid. Otherwise the method returns 0.

Return value (long)
Number of grid lines.

SetCurrentGridCell(long Row, long logColumn)

Sets the cursor in a specified cell in a grid.
The element must be the current element in the window, otherwise a runtime error is generated.

Parameter

Row	Line in grid, beginning with "1"
logColumn	Logical column number (independent of any user-specific column ordering), beginning with "1" Each column has a unique logical number that is maintained independent of the position ordering in the grid.

Return value (VARIANT_BOOL)
When the grid cell could be set, the result is TRUE, otherwise a runtime error is generated.

GetCurrentGridCell(VARIANT Row, VARIANT logColumn)

Returns in the pass parameters the current logical position of the cursor in a grid.

Parameter

Row	Current grid line, beginning with "1"
logColumn	Logical column number (independent of any user-specific column ordering), beginning with "1" Each column has a unique logical number that is maintained independent of the position ordering in the grid.

Return value (VARIANT_BOOL)

When the grid cell was found, the result is TRUE, otherwise a runtime error is generated.

GetGridCellValue(long Row, long logColumn)

Returns the values of the specified cell in a grid. A runtime error is generated when an invalid or empty cell is queried. The value of the current grid cell can also be read with the **Contents** property of the **FGControl** object.

Parameter

Row	Current grid line, beginning with "1"
logColumn	Logical column number (independent of any user-specific column ordering), beginning with "1" Each column has a unique logical number that is maintained independent of the position ordering in the grid.

Return value (VARIANT)
Value from grid cell.

Validate

This method validates the current element (corresponds to the user pressing the ENTER key when the focus is on an element) and moves the cursor to the next element. Within a grid, this method is only executed for cells that contain an edit field or combo box (only these fields require validation).

Refresh

This method refreshes values in the element on the screen. This may be needed when the variable value that is connected with a window eleemt has been changed.

SetGridColReadOnly(long logColumn, VARIANT bSet)

This function sets the specified column to read only or removes the read only status.

Parameter

logColumn	Grid column that should be changed, i.e., the 'logical' column starting with "1"
bSet	Optional parameter (by default TRUE) that specifies whether column is set to read only or not

Return value (VARIANT_BOOL)
When the read only status can be set, TRUE is returned, otherwise FALSE.

GetGridColReadOnly(long logColumn)

This function checks whether the specified column is set to read only or not.

Parameter

logColumn	Grid column that should be changed, i.e., the 'logical' column starting with "1"

Return value (VARIANT_BOOL)
When the read only status is set, TRUE is returned, otherwise FALSE.

5.8. CwlPreview

An object of this class represents a CWL Preview. A Preview is a special version of a control (CWLFgControl), which can be accessed through its **Preview** property. A Preview consists of a collection of **PreviewPages**, which in turn contains a collection of **PreviewPageItems**.
The ID of a Preview, with which the Preview can be referenced from the **CWLFgCollection**, is dynamic and must be determined during runtime.
A simple Preview window consists of one window with exactly one PreviewControl.

Exposed in

☐ System Macros
☐ CTK Macros

CWLPreview
Properties
long **PageCount**
long **CurrentPageNr**
ICwlPreviewPage* **Page** (long nPageNr)
Print (BOOL bChoosePrinter)
Mail (BOOL bWithDialog)

5.8.1. Properties

PageCount [long, read only]
Number of pages (**CWLPreviewPage** objects) in the Preview (**Page** property).
If the preview is being filled, it may happen that this value does not yet contain the final number of pages.

CurrentPageNr [long, read only]
Current page that is being displayed in this Preview.

5.8.2. Methods

Page(long nPageNr)
Returns an object of the **CWLPreviewPage** class that represents the page specified with the number contained in the **nPageNr** parameter.

Parameter

nPageNr	Number of page, beginning with "1"

Return (ICwlPreviewPage*)
Pointer to an object in class **CWLPreviewPage** for the corresponding page.

Print(BOOL bChoosePrinter)
This method prints the current document on the printer/spooler. When TRUE is passed with parameter bChoosePrinter, the printer selection dialogue displayed, where the printer for the output can be chosen. When FALSE is passed with parameter bChoosePrinter, the output is printed to the standard printer. When output is being steered to the spooler, the document will be printed to the spooler.

Parameter

bChoosePrinter	Show select printer dialogue

Return value (none)

Mail(BOOL bWithDialog)

This method sends the current document as an email in the currently set email attachment file format. When TRUE is passed with parameter bWithDialog, an email dialogue window is opened for entry of recipient and other optional data for the transmission. When FALSE is passed with parameter bWithDialog, an AUX:MAIL control element must be contained in the form for the document, from which the email recipient is obtained. When no AUX:MAIL is found, an error will be generated.

Parameter

bWithDialog	Open dialogue window for entry of email recipient and other optional transmission data

Return value (none)

5.9. CwlPreviewPage

An object of this class represents an individual page in a Preview. It consists of a sequence of PreviewPage objects of the **CWLPreviewPageItem** class.

Exposed in

☐ System Macros
☐ CTK Macros

CWLPreviewPage
Properties
long **ItemCount**
ICwlPreviewPageItem* **Item** (long nIndex)

5.9.1. Properties

ItemCount [long, read only]
Number of elements of type **CWLPreviewPageItem** in this PreviewPage.

5.9.2. Methods

Item(long nIndex)
Returns an individual element in this PreviewPage of the **CWLPreviewPageItem** class.

Parameter

nIndex	Number of elements in PreviewPage (**0 to ItemCount-1**)

5.10. CwlPreviewPageItem

This object represents an individual element within a PreviewPage (**CWLPreviewPage**).

Exposed in

☐ System Macros
☐ CTK Macros

CWLPreviewPageItem
Properties
BSTR **Text**
long **View**
long **Var**
long **Line**
long **Column**
CWLSpoolItemType **Type**
CWLAlignements **Alignment**

long **Font**
long **Height**
long **Width**
BSTR **HiddenText** (CWLSpoolPreviewItemFlag flag)

5.10.1. Properties

Text [BSTR, read only]
Content of element.

View [long, read only]
The view that is assigned to the element (saved in PDF Editor).

Var [long, read only]
The var that is assigned to the element (saved in PDF Editor).

Line [long, read only]
The line in which the element is displayed

Column [long, read only]
The column in which the element is displayed.

Type [long, read only]
Type of preview elements (see CWLSpoolItemType)

Alignment [long, read only]
Alignment of element (see CWLAlignments).

Font [long, read only]
Font of element.

Height [long, read only]
Height of element.

Width [long, read only]
Width of element.

HiddenText (CWLSpoolPreviewItemFlag flag) [BSTR, read only]
Returns the text that is assigned to the flag (see CWLSpoolPreviewItemFlag). Normally only DrillDown elements contain a text.

5.11. CwlSpreadSheet

CWLSpreadSheet
Properties
BSTR Contents
long LineCount
long ColumnCount
BSTR Formula
Methods
BOOL **SetCurrentCell** (long row, long col)
BOOL **GetCurrentCell** (VARIANT *row, VARIANT *col)
Recalc
Redraw
BOOL **ExportAsXLS** (BSTR NameAndPath)
BOOL **Load** (BSTR NameAndPath)
BOOL **Save** (BSTR NameAndPath)

5.11.1. Properties

Contents [BSTR]
Contents of the current cell in a spread sheet.

LineCount [long]
Number of lines in the current spread sheet.

ColumnCount [long]
Number of columns in the spread sheet.

Formula [BSTR]
The formula in the current line. When the line contains a constant value, an empty text is returned.

5.11.2. Methods

SetCurrentCell (long row, long col)
Sets the focus on a desired cell.

Parameter

Row	Line in spread sheet
Col	Column in spread sheet

Return value BOOL)
Returns FALSE when the specified cell does not exist, otherwise TRUE.

GetCurrentCell (VARIANT *row, VARIANT *col)

Determines the current cell in a spreadsheet and sets the passed parameters row und col to the corresponding values.

Parameter

row	Reference to a variable that is filled with the line
col	Reference to a variable that is filled with the column

Return value (BOOL)
Returns FALSE when the spread sheet has not been clicked on, i.e., no cell is active.

Recalc

Forces the spreadsheet to recalculate values.

Redraw

Forces a screen refresh of the spreadsheet.

SaveAsXLS (BSTR NameAndPath)

Exports a spreadsheet in XLS format. Thereby all formulas and settings are lost which are not compatible with MS Excel.

Parameter

NameAndPath	Name of target file with path

Return value (BOOL)
Returns false when the spread sheet could not be exported.

Load (BSTR NameAndPath)

This method loads a previously saved spread sheet (see Save). The state of the spread sheet as it was saved is exactly reproduced.

Parameter

NameAndPath	Name of target file with path.

Return value (BOOL)
Returns FALSE when the spread sheet could not be loaded.

Save (BSTR NameAndPath)

Exports the spread sheet in the internal format. This format can be imported later on (see Load) and the spread sheet is reproduced in the exact state in which it was saved.

Parameter

NameAndPath	Name of target file with path.

Return value (BOOL)
Returns FALSE when the spread sheet could not be saved.

1.2. CWLGrid

CWLGrid

Properties

VARIANT Contents
long LineCount
long ColumnCount
BOOL IsRedraw

Methods

BOOL **SetCurrentCell** (long row, long col)
BOOL **GetCurrentCell** (VARIANT *row, VARIANT *col)
BOOL **ExportAsXLS** (BSTR NameAndPath)
BOOL **Load** (BSTR Settings)
BOOL **Save** (BSTR Settings)
VARIANT **GetCellValue** (long row, long col)
BOOL **GetColumnReadOnly** (long col)
SetColumnReadOnly (long col, VARIANT bSet, VARIANT bRedraw)
long **AddColumn** (BSTR ColumnTitle, BSTR ColumnControl, BSTR align, BSTR Type, int Font, int View, int Var, int ColWidth, VARIANT AddFlags, VARIANT ColumnColor, VARIANT bRedraw)
BOOL **RemoveColumn** (long col, VARIANT bRedraw)
SetColumnColor (long col, RGB color)
RGB **GetColumnColor** (long col)
SetLineColor (long line, RGB color)
RGB **GetLineColor** (long line)
BOOL **MoveColumn** (long col, long Position)
BOOL **SetColumnWidth** (long col, long Width)
long **GetColumnWidth** (long col)
long **GetLogColumn** (long ColumnOnScreen)
long **GetPhysColumn** (long col)
SetComboStrings (long col, BSTR theStrings)
Validate
Refresh
BOOL **IsUserColumn** (long col)
BOOL **Header**
BOOL **Footer**
BOOL **AddLine**
BOOL **RemoveLine** (long line)
BOOL **InsertLine** (long line)
BOOL **ReplaceLine** (long line)
GetLineValues (long line)
BOOL **InitUserGrid**
BOOL **SetColumnTitle** (long line, long col, BSTR Text)

1.2.1. Properties

Contents [VARIANT]
The content of the current cell.

LineCount [long]

Number of lines.

ColumnCount [long]
Number of columns.

IsRedraw [BOOL]
Status whether changes in the screen grid display are refreshed immediately (is set to make several grid changes and then refresh the grid display afterwards).

1.2.2. Methods

BOOL SetCurrentCell (long row, long col)
Sets the focus to the desired cell.

Parameter

Row	Line in the screen grid
Col	Logical column number

Return value (BOOL)
Returns FALSE when the specified cell does not exist, otherwise TRUE.

BOOL GetCurrentCell (VARIANT *row, VARIANT *col)
Sets the current cell in the screen grid and sets the parameters for row and col to the passed values.

Parameter

Row	Reference to a the row value of the cell value to be accessed
Col	Reference to a the column value of the cell value to be accessed

Return value (BOOL)
Returns FALSE when the current cell could not be identified due to an internal error.

BOOL ExportAsXLS (BSTR NameAndPath)
Exports the screen grid in XLS format.

Parameter

NameAndPath	Name of output file with path.

Return value (BOOL)
Returns FALSE when the screen grid contents could not be exported.

BOOL Save (BSTR Settings)
Saves the entire settings of the screen grid under a name that is specified with parameter **Settings**. Already existing settings data with the same name will be overwritten.

Parameter

Settings	Name of the saved settings

Return value (BOOL)
Returns FALSE when the settings could not be saved.

BOOL Load (BSTR Settings)

Loads the entire contents of the screen grid that has been saved under the name in parameter **Settings**.

Parameter

Settings	Name of saved settings

Return value (BOOL)
Returns FALSE when the settings could not be loaded.

VARIANT GetCellValue (long row, long col)

Returns the value at **row** (line) and **col** (column) in the grid.

Parameter

row	Screen grid row
col	Logical column number

Return value (VARIANT)
Value that is saved at the spedified position in the screen grid.

BOOL GetColumnReadOnly (long col)

Returns the "read only" status of the specified column (**col**).

Parameter

col	Logical column number

Return value (BOOL)
Returns FALSE or TRUE.

SetColumnReadOnly (long col, VARIANT bSet, VARIANT bRedraw)

Column **col** will be set to "read only". Columns that are set to read only are displayed in a separate color and can no longer be selected.

Parameter

Col	Logical column number
bSet	Read only is set or not (value = TRUE when parameter not specified)
bRedraw	Changes are displayed immediately on the screen (value = TRUE when parameter not specified)

long AddColumn (BSTR ColumnTitle, BSTR ColumnControl, BSTR align, BSTR Type, int Font, int View, int Var, int ColWidth, VARIANT AddFlags, VARIANT ColumnColor, VARIANT bRedraw)

This method adds a new column at the end of the screen grid. A screen grid can have a maximum of 199 columns. When the limit is reached, no new columns can be inserted.

Parameter

ColumTitle	Column title
ColumnControl	Text that describes the control that is displayed in the cell (see the table of eligible controls below).
align	Alignment of column:

	l... align left r... align right z... centered
Type	Cell type: T... plain display text V... Variable that is displayed in the cell G... graphic
Font	The font combination number (allowed values from 0 to 9). The fonts can changed in CWLCTK in menu item 'Edit Mesonic Fonts' in tab area 'Other Fonts'.
View	Table (or 0) from which the displayed variable is loaded.
Var	Number of variable in the **View**.
ColWidth	Column width in screen units.
AddFlags	A combination of values that controls various column properties: SORTFLAG = 1 (sortable column) HIDEFLAG = 4 (column can be hidden) READONLYFLAG = 8 (column is read only) MOVEFLAG = 16 (column position can be moved) SIZEFLAG = 32 (column size can be changed) INVISIBLEFLAG = 64 (column is invisible) COMPANYYEARFLAG = 256 (column contains the fiscal year and is automatically converted when used by another calendar) This value is optional and is 0 when not specified.
ColumnColor	Use to set a background color for column. When not specified, the column contains no color (except when it is read only).
Redraw	Optional parameter. Default value = TRUE when not specifed. Value speciffies whether changes are displayed on the screen immediately, or whether the **Refresh** method must be called when this value is set to FALSE.

Eligible controls for a cell in a screen grid control (requires column type setting "V"):

Type	Control	Variable type	Supported Parameters	Example
T1	Entry field	Text	Zx (maximum number of chars) L1 (Match code icon) Ox (Object type)	„T1,Z10,L1,my entry field"
T2	Entry field	Integer	Zx (maximum number of chars) Ox (Object type)	„T2,Z5,my entry field"
T3	Entry field	Double	Zx (maximum number of chars) Ix (Number of decimal places)	„T3,Z15,I2,L1,my entry field"

			L1 (Matchcode icon) 0x (Object type)	
T5	Entry field	Capital letters	Zx (maximum number of chars) L1 (Match code icon) 0x (Object type)	„T5,Z10,L1,my entry field"
T6	Entry field	Date	Zx (maximum number of chars) L1 (Match code icon) 0x (Object type)	„T6,Z10,L1,my entry field"
T12	Checkbox	Text	Z1 0x (Object type)	„T12,Z1,my check box"
T17	Read only checkbox	Text	0x (Object type)	„T12,Z1,my check box"
T21	Static	Text	0x (Object type)	„T21,my value*
T22	Static	Integer	0x (Object type)	„T22,my value*
T23	Static	Double	0x (Object type)	„T23,my value *
T25	Static	Capital letters	0x (Object type)	„T25,my value*
T26	Static	Date	0x (Object type)	„T26, my value*
T31	Combo box	Text	Zx (Entry length) Lx (list box width, with 0 the grid column width is used) Hx (list box height)	„T31,Z1,L0,H5,my combo box"
T32	Read only combo box	Text	Zx (entry length)	„T32,Z1,my read only combo box"

* Static values can often be better represented by not specifying the type (e.g., T21), but rather by formating the output, similar use in the CWLPDFE Editor (e.g. "{DATETIME}" with a date).

Return value (long)
Logical number of the inserted column, or 0 when the insertion was unsuccessful.

BOOL RemoveColumn (long col, VARIANT bRedraw)

Removes an inserted user-defined column. Standard grid columns cannot be removed.

Parameter

col	Die logische Spaltennummer
Redraw	Optional parameter. Default value = TRUE when not specifed. Value speciffies whether changes are displayed on the screen immediately, or whether the **Refresh** method must be called when this value is set to FALSE.

Return color (BOOL)
Returns FALSE or TRUE, depending on whether the column was removed or not.

SetColumnColor (long col, RGB color)

Column **col** is set to color **color**. The value is specified as RGB value.

Paramter

col	Logical column number
color	Desired RGB color (VBScript has the RGB function: RGB(Red value, Green value, Blue value))

RGB GetColumnColor (long col)

This method returns the RGB color of column **col**. When no color is set, -1 is returned as value.

Parameter

col	Logical column number

Return value (RGB)
Column background color.

The red/green/blue values of the current column color can be determined with the following function:

```
If (color >= 0) Then
      blue = color\65536
      green = (color-(blue*65536))\256
      red = color-(blue*65536) - (green*256)
End If
```

SetLineColor (long line, RGB color)

Grid row **line** is set to color **color**. The color is passed in RGB format.

Parameter

line	Logical column number
color	Desired RGB color (VBScript has the RGB function: RGB(Red value, Green value, Blue value))

RGB GetLineColor (long col)

The color of the line Es wird die Farbe der Zeile **line** im RGB-Format zurückgegeben. Ist keine eigene Farbe gesetzt, wird -1 zurückgegeben.

Paramter

col	Die logische Spaltennummer

Rückgabewert (RGB)
Die Farbe der Spalte. Die rot/blau/grün - Komponenten des Farbwerts kann mit folgender Funktion bestimmt werden:

```
If (color >= 0) Then
      blue = color\65536
      green = (color-(blue*65536))\256
      red = color-(blue*65536) - (green*256)
End If
```

BOOL MoveColumn (long col, long Position)

Moves the specified column (**col**) to position **Position**.

Parameter

col	Logical column number
Position	Target position (1 to number of columns) to which the column should be moved.

Return value (BOOL)
Returns FALSE or TRUE.

BOOL SetColumnWidth (long col, long Width)

Changes the column width of column **col** to value **Width** (in screen units).

Parameter

col	Logical column number
Width	Column width in screen units (font-dependent unit). When 0 is passed, the column is effectively „hidden".

Return value (BOOL)
Returns FALSE or TRUE.

long GetColumnWidth (long col)

This function returns the width of column **col** in screen units.

Parameter

col	Logical column number

Return value (long)
Width of column in screen units.

long GetLogColumn (long ColumnOnScreen)

This function returns the logical column number of the screen grid column **ColumnOnScreen.**

Parameter

ColumnOnSceen	The column position on the screen

Return value (long)
Logical column number.

long GetPhysColumn (long col)

This function returns the position of column **col** on the screen.

Paramter

Col	Logical column number

Return value (long)
Column position on the screen.

SetComboStrings (long col, BSTR theStrings)

This function is used to set combo box selection options for columns that contain a combo box control. The entries are passed in a single string, whereby the individual selection entries are separated by line breaks. Is the combo box is defined without an entry length, the texts only have to be separated with CR/LF (the entry length specifies the length of the combo box value. This value is saved in the variable that is displayed in the combo box. The display text is a descriptive text for the actual value).

Parameter

col	Logical column number
theStrings	String with combo box selection options.. The string is composed of entries formatted as:

	Value<Tab>Display text<CR><LF> Value<Tab>Display text<CR><LF> ...

Example:

```
combostring = "0"&chr(9)&"Option 0"&chr(13)&chr(10)
combostring = combostring & "1"&chr(9)&"Option 1"&chr(13)&chr(10)
combostring = combostring & "2"&chr(9)&"Option 2"&chr(13)&chr(10)
myGrid.SetComboStrings 14, combostring
```

Validate

When the text of a cell is changed (e.g., with a macro command), you can use this method to trigger the validation check of the entered value, which is also triggered by the **OnGridCheckUserColumn** event.

Refresh

This function causes a refresh of the screen display of the grid.

BOOL IsUserColumn (long col)

This function determines whether column **col** is a user-defined column that has been inserted by script to the grid.

Parameter

col	Logical column number

Return value (BOOL)
TRUE / FALSE

BOOL Header

This method outputs the header area of the grid. The function is only supported in user-defined grid controls.

BOOL Footer

This method outputs the footer area of the grid. The function is only supported in user-defined grid controls.

BOOL AddLine

This function inserts a new row at the end of the grid. The columns receive the values that are saved in the associated variables. The variables that are displayed in a grid are determined by the column definition. The function is only supported in user-defined grid controls.

Return Value (BOOL)
FALSE when insertion is unsuccessful, otherwise TRUE.

BOOL RemoveLine (long line)

This function removes the row in number **line**.
The function is only supported in user-defined grid controls.

Parameter

line	Row number

Return value (BOOL)

FALSE when removal is unsuccessful, otherwise TRUE.

BOOL InsertLine (long line)

This inserts a new row before the grid row specified with number **line**. The columns receive the values that are saved in the associated variables. The variables that are displayed in a grid are determined by the column definition. The function is only supported in user-defined grid controls.

Parameter

line	Row number

Return value (BOOL)
FALSE when removal is unsuccessful, otherwise TRUE.

BOOL ReplaceLine (long line)

This function replaces the row with number **line** with a new row. The columns receive the values that are saved in the associated variables. The variables that are displayed in a grid are determined by the column definition. The function is only supported in user-defined grid controls.

Parameter

line	Row number

Return value (BOOL)
FALSE when unsuccessful, otherwise TRUE.

GetLineValues (long line)

This function copies the respective column values of the row with number **line** into the variables associated with the columns.
The function is only supported in user-defined grid controls.

Parameter

line	Row number

BOOL InitUserGrid

This function initializes the grid object and connects the variables of the window with the grid.
The grid object can first be used, when this function has been successfully executed.
The function is only supported in user-defined grid controls!

Return value (BOOL)
FALSE when unsuccessful, otherwise TRUE.

BOOL SetColumnTitle(long line, long col, BSTR Text)

This function is used to change the text in a column header.
When the element is connected to a variable, the text may not be longer than the length of the variable. Otherwise, the display will be truncated to the variable length. The new text is copied into the variable.

Parameter

line	Line number, in the header normally line 1
Col	Logical column of the header text to be changed.
Text	Text that will be displayed in the column header

Return value (BOOL)
FALSE when unsuccessful, otherwise TRUE.

Example:

This example is for a modified window that contains a grid with control element ID 100.
Using several buttons, the following actions are performed in the grid:
- a column is inserted, it is set to a color and moved to grid screen position 3.
- the column is removed
- the grid settings are saved and loaded again
- the grid contents are saved to an XLS chart
- the basic grid properties are obtained

```
' Event, that is fired when a user grid column is exited
Sub CWLCurrentWindow_OnGridCheckUserColumn(nFgId, nRow, nColumn, bResult)
      If nFgId = 100 Then
            Set myGrid = CWLCurrentWindow.ActiveWindow.Controls.Item(100).Grid

            ' do not use value "2" in rows <= 5
            If myGrid.Contents = "2" And nRow <= 5 Then
                  General.MsgBox "Only values 0 and 1 are allowed in rows 1 to
                  5!"
                  bResult.value = False
            End If
      End If
End Sub

' Push button events
Sub CWLCurrentWindow_OnPushButton(nFgId, bResult)

      Dim row, column
      Set myGrid = CWLCurrentWindow.ActiveWindow.Controls.Item(100).Grid

      If nFgId = 800 Then ' Push button 'Insert column'
            If myGrid.ColumnCount = 13 Then ' column is not inserted yet?
                  myGrid.isRedraw = 0 ' do not show changes

                  ' the new column is a a combo box with entry length 1
                  ' Create the list of combo box selection options
                  combostring = "0"&chr(9)&"Option 0"&chr(13)&chr(10)
                  combostring = combostring & "1"&chr(9)&"Option
1"&chr(13)&chr(10)
                  combostring = combostring & "2"&chr(9)&"Option
2"&chr(13)&chr(10)
                  myGrid.SetComboStrings 14, combostring

                  ' Create new variables for the grid column in the user-defined
                  variables
                  CWLCurrentWindow.ActiveWindow.Vars.CreateVar 495, 0, "1", 1,
"1"

                  ' Insert column
                  myColumnNumber = myGrid.AddColumn ("My column",
"T31,Z1,L30,H3,mycombo","1", "V", 0, 495, 0, 20)

                  ' Set column to third position
                  myGrid.MoveColumn myColumnNumber,3
```

```
                ' Change column color of new column
                myGrid.SetColumnColor myColumnNumber, RGB(177, 200, 233)
                ' Change row color of lines 5,7,9 and 11
                myGrid.SetLineColor 5, RGB(222, 232, 245)
                myGrid.SetLineColor 7, RGB(222, 232, 245)
                myGrid.SetLineColor 9, RGB(222, 232, 245)
                myGrid.SetLineColor 11, RGB(222, 232, 245)

                myGrid.isRedraw = 1 ' refresh grid display

                ' !! this can first be performed after isRedraw = 1 since
                ' no controls are created during the suppress Redraw
                ' and the inserted combobox would therefore not be present yet
                ' This would prevent a successful SetContents
                ' Set focus to the line 3 in the new column
                myGrid.SetCurrentCell 3, myColumnNumber

                ' Set cell value to "2" ==> provokes an error with
                OnGridCheckUserColumn
                myGrid.Contents = "2"

        End If
    End If

    If nFgId = 799 Then     ' Button 'Remove column'
        If myGrid.ColumnCount = 14 Then
            myGrid.RemoveColumn 14
        End If
    End If

    If nFgId = 798 Then     ' Button 'Load Settings'
        myGrid.load "MDP Settings"
    End If

    If nFgId = 797 Then ' Button 'Save settings'
        myGrid.save "MDP Settings"
    End If

    If nFgId = 796 Then     ' Button 'Export to Excel'
        myGrid.ExportAsXLS "c:\mdp script grid.xls"
    End If

    If nFgId = 795 Then     ' Button 'Analyze Grid Infos'
        msg = "Grid Information:" & chr(13) & chr(10)
        msg = msg & "Rows: " & myGrid.LineCount & chr(13) & chr(10)
        msg = msg & "Columns: " & myGrid.ColumnCount & chr(13) & chr(10)
        myGrid.GetCurrentCell row, column
        msg = msg & "Current cell: " & row & "/" & column & chr(13) &
chr(10)
        msg = msg & "Cell contents: " & myGrid.Contents & chr(13) & chr(10)
        msg = msg & "logical column number: " & column & chr(13) & chr(10)
        msg = msg & "Visible column number: " & myGrid.GetPhysColumn
(column) & chr(13) & chr(10)
        col = myGrid.GetColumnColor (column)
        colstr = "<not set>"
        If (col >= 0) Then
            blue = Clng(col\65536)
```

```
                  green = Clng((col-(blue*65536))\256)
                  red = col-(blue*65536) - (green*256)
                  colstr = col & "= red: "&red&", green: "&green&", blue: "&blue
            End If
            msg = msg & "current column color: " & colstr & chr(13) & chr(10)
            col = myGrid.GetLineColor(row)
            colstr = "<not set>"
            If (col >= 0) Then
                  blue = col\65536
                  green = (col-(blue*65536))\256
                  red = col-(blue*65536) - (green*256)
                  colstr = col & "= red: "&red&", green: "&green&", blue: "&blue
            End If
            msg = msg & "current row color: " & colstr & chr(13) & chr(10)
            msg = msg & "Current row is read/only: " & myGrid.GetColumnReadOnly
(column) & chr(13) & chr(10)
            msg = msg & "current column width: " & myGrid.GetColumnWidth
(column) & chr(13) & chr(10)
            msg = msg & "Redraw active: " & myGrid.isRedraw & chr(13) & chr(10)
            general.MsgBox msg
      End If

End Sub
```

1.3. CWLReport

CwlReport
Properties
BSTR Name
short Type
BSTR HeaderFLags
BSTR MiddleFlags
short MultilinesLeft
BSTR Title
BSTR Description
BOOL ShowAbortWin
long Id
BOOL EnableDrilldown
Methods
BOOL **Header** (VARIANT Flags)
short **Middle** (VARIANT Flags)
BOOL **Footer** (VARIANT Flags)
void **SetHiddenText** (short Type, BSTR Text, VARIANT where)
Events
OnPrintDrildownItem (int ReportId, CWLEventResult DrillDownText, short View, short Var, BSTR ItemText)
OnCancel (int ReportId, CWLEventResult MayClose)
OnDrillDown (int ReportId, BSTR DrilldownText, BSTR Text)

1.3.1. Properties

Name [BSTR]

Name of report.

Type [short]
Type of output for the report:
1... to the screen
2... to the printer
4... to the spooler

HeaderFlags [BSTR]
The "flags" that are currently active for the header and footer.

MiddleFlags [BSTR]
The "flags" that are currently active for the form middle section.

Title [BSTR]
Form title (name) (max. 50 characters) that is saved in the spool file.

MultilinesLeft [short]
Number of lines of a multi-line text that will be printed on next page due to page break.

Description [BSTR]
The report description (max. 100 characters) that is saved in the spool file and is shown for example in the grid of printed documentsn in the Despooler window.

ShowAbortWin [BOOL]
This property determines whether a small window wih the printing progress is displayed, including an option to cancel the print out.

Abbildung: Anzeige des „AbortWin" beim Drucken

Id [long, read only]
This property is a number that is unique for each report while the program is running.

EnableDrillDown [BOOL]
When the property is set to TRUE, drill-down entries are active for mouse-clicks.

1.3.2. Methods

BOOL Header (VARIANT Flags)
The header section of the report description is printed. When the **Flags** parameter is passed, the flags are used for printing, are saved in *CwlReport* and can be obtained with the *HeaderFlags property*.

Parameter

Flags	Flags to be used (optional parameter)

Return value (BOOL)

Returns FALSE when printing was cancelled by an error.

short Middle (VARIANT Flags)

The middle section of the report description is printed with this method. When the **Flags** parameter is passed, the flags are used for printing, are saved in *CwlReport* and can be obtained with the *MiddleFlags* property.

Parameter

Flags	Flags to be used (optional parameter)

Return value (short)
0... output was executed
1... error occurred
2... output continued on next page, not enough room on current page.
When a multiline text has been printed, it could happen that the values in the middle section were printed, but the text was partially printed on the next page with a page break. This can be tested with property *MuliLinesLeft* . In this case, the next variables can be set for the first middle section on the next page, the split off multiline lines are then automatically printed at the start of the new page.

BOOL Footer (VARIANT Flags)

This method outputs the footer of the report description. When the **Flags** parameter is passed, the flags are used for printing, are saved in *CwlReport* and can be obtained with the *HeaderFlags* property.

Parameter

Flags	Flags to be used (optional parameter)

Return value (BOOL)
Returns FALSE when printing was cancelled due to an error.

void SetHiddenText (short Type, BSTR Text, VARIANT where)

Für die Ausgabe von Kopf, Mittelteil und Fuß kann jeweils ein bestimmter Text für den Hiddentext für Drilldown-Elemente gesetzt werden. Der „Hiddentext" ist der versteckte Text, der das Ziel des Drilldown-Elements darstellt (im Gegensatz zum angezeigten Text). Dieser Text wird dann in jedem Drilldown-Element des Ausgabebereichs verwendeet.

Paramter

type	Der Typ des „Hiddentext". Der Typ sollte immer 0 sein.
Text	Der Text, der im „Hiddentext" gespeichert wird
where	Wo wird der „Hiddentext" gesetzt 0... Kopf 1... Mittelteil 2... Fuß

1.3.3. Events

OnPrintDrildownItem (int ReportId, CWLEventResult DrillDownText, short View, short Var, BSTR ItemText)

Event is fired when printing an entry that is coded as a drill-down element. This assigns a separate "Hiddentext" for each drill-down element.

Parameter

ReportId	Report ID for which the report is fired (each report has an unique ID)
DrillDownText	The text that is set in the drill-down element currently and that can be changed.
View	The table that contains the printed value
Var	The variable in the table that is printed with the entry
ItemText	Text of the printed entry

OnCancel (int ReportId, CWLEventResult MayClose)

This event is fired when the STOP button is pressed in the report or the report window is closed ("red x"). When printing is done to the printer, and the *ShowAbortWin* property is set, the event is also fired when the STOP button is clicked.
The STOP button can be clicked while printing is being performed. When supported in the program, printin can be cancelled during output (a global variable must be defined that is checked in the report printing loop and that is set in the event handler).

Parameter

ReportId	Report ID for which the report is fired (each report has an unique ID)
MayClose	When the value is set to TRUE (MayClose.Value = true), the report is closed.

OnDrillDown (int ReportId, BSTR DrilldownText, BSTR Text)

This event is fired when the user clicks on a drill-down element. The *EnableDrillDown* property must be set to TRUE for this purpose.

Parameter

ReportId	Report ID for which the report is fired (each report has an unique ID)
DrillDownText	The text that is currently set in the drill-down element.
Text	The visible text of the entry

6. Constants

6.1. CWLApplicationNr

ID for applications that can be active.

Name	Wert
cwlMAIN	0
cwlFIBU	1
cwlFAKT	2
cwlLOHN A	3
cwlLIST	4
cwlKORE	5
cwlANBU	6
cwlINFO	11
CwlLOHN D	18
cwlPROD	20

6.2. CWLWindowTypes

Type of window. See **Type** property of objects of the **CWLWindow** class.

Name	Value	Description
winStandardType	0	Standard CWL window
winPreviewType	1	Preview window
WinScriptType	2	UserForm of a VB script

6.3. CWLControlTypes

Type of an element (control) in a window. See **Type** property of objects of the **CWLFgControl** class.

Name	Value	VarType	Description
cwlControlEditString	1	8	
cwlControlEditInteger	2	3	
cwlControlEditFloat	3	5	
cwlControlEditDouble	4	5	
cwlControlEditUppercase	5	8	
cwlControlEditDate	6	7	
cwlControlEditMultiline	7	8	
cwlControlEditPassword	8	-	No Content
cwlControlEditTimespan	9	3	
cwlControlButton	11	-	No Content
cwlControlCheckbox	12	8	"1"=on "0"=off
cwlControlRadioButton	13	8	"1" for group element that is selected (the other "0")
cwlControlListbox	15	-	
cwlControlTree	18	-	

cwlControlStaticString	21	-	No Content
cwlControlStaticInteger	22	-	No Content
cwlControlStaticFloat	23	-	No Content
cwlControlStaticDouble	24	-	No Content
cwlControlStaticUppercase	25	-	No Content
cwlControlStaticDate	26	-	No Content
cwlControlStaticTimespan	29	-	No Content
cwlControlFrame	30	-	No Content
cwlControlCombobox	31	-	No Content
cwlControlGrid	35	-	No Content
cwlControlPreview	36	-	No Content
cwlControlSpreadsheet	37	-	No Content

6.4. CWLSpoolItemType

Type of elements in a Spool Preview. See **Type** property of objects of the **CWLPreviewPageItem** class.

Name	Value	Description
cwlSpoolItemText	84	Text constant as saved in form
cwlSpoolItemVar	86	variable Text that is filled during print out with variable value
cwlSpoolItemLookup	85	'Lookup' element that searches for a text in a variable in another table and returns the value found there
cwlSpoolItemGraphic	71	Graphics element (bitmap)
cwlSpoolItemObject	79	chart element (bar, pie or line)
cwlSpoolItemBar	66	Graphical percentage display as horizontal bar
cwlSpoolItemControl	83	Element contains non-printed commands that are executed during print out (e.g., printer change)
cwlSpoolItemFormula	70	Formula that is executed during print out
cwlSpoolItemLine	76	horizontal or vertical line
cwlSpoolItemRect	82	Rectangle (filled or not)
cwlSpoolItemMultiline	77	Text that is shown in defined width but variable height carriage return)

6.5. CWLSpoolPreviewItemFlag

Type of hidden text element in a **CwlPreviewPageItem**.

Name	Value	Description
cwlHiddenflagDrilldown	0	A text can be assigned to drill down items
cwlHiddenflagGroup	1	GroupItems do not normally have a text assigned
cwlHiddenflagEdit	2	Items that are edited (e.g. in Quick Entry) do not normally have a text assigned
cwlHiddenflagUser	3	Item can have a text assigned

6.6. CWLAlignments

Type of elements in a Spool Preview. See **Alignment** property of objects in the **CWLPreviewPageItem** class.

Name	Value	Description
cwlAlignLeft	108	Left
cwlAlignRight	114	Right
cwlAlignCenter	122	Centered

6.7. CWLScriptWindowType

Type of system script. See **Parameter** mode of the **CWLStart.RunFormScript** method.

Name	Value	Description
cwlScriptWindowStandard	0	As with all normal program windows (and CTK windows when not started modally),l the window is hidden at module change
cwlScriptWindowModal	1	Modal window, the application can first be started after the window is closed
cwlScriptWindowSystem	2	Script window that remains visible above other windows and remains visisble after module change. Windows of this type have no internal ID and cannot be identified in the MayCloseWindow event.

6.8. CWLSystemServerType

Type of system database. See *what* Parameter in method CWLCompany.GetSystemConnection.

Name	Value	Description
cwlSystemServerSRV	2	System database (database connections, users, user rights, MSM, audit, user groups, etc.)
cwlSystemServerARC	4	System database with archive data tables
cwlSystemServerPDB	8	System database with form and window description data
cwlSystemServerCMP	16	System database for company independent data
cwlSystemServerLOHN	32	System database for Austrian payroll data
cwlSystemServerLOHD	64	System database for German payroll data

6.9. CWLDbConnectionType

Type of database connection.

Name	Value	Description
cwlDbConnectionTypeDAO	0	Microsoft Access Database format
cwlDbConnectionTypeSQL	1	Microsoft SQL server database format
cwlDbConnectionTypePOS	4	PostgreSQL database format Attention: PostgreSQL is no longer supported from Winline Version 8.6!